

SYSTEM DESIGN INTERVIEW

Design Uber (Ride Hailing)

real-time matching of riders and drivers, geo-indexed dispatch, trip lifecycle, surge pricing

~ 45 minute read

© Study Guide — For personal use only

What's Inside

- 01 · Problem Statement
- 02 · Requirements
- 03 · Capacity Estimation
- 04 · High-Level Architecture
- 05 · Geo-Indexing with H3
- 06 · Driver Location Pipeline
- 07 · Disco: Matching & Dispatch
- 08 · Trip Lifecycle & State Machine
- 09 · Surge Pricing
- 10 · ETA & Routing Engine

- [11](#) · [Storage & Sharding](#)
- [12](#) · [Failure Modes & Recovery](#)
- [13](#) · [Design Trade-offs](#)
- [14](#) · [Scalability & Distribution](#)
- [15](#) · [Interview Playbook](#)

01

Problem Statement

Requirements and scope

Design a global ride-hailing service like **Uber** or **Lyft**. The system must match millions of riders with nearby drivers in real time, dispatch trips with low latency, track the trip lifecycle from request to payment, and dynamically price rides based on supply and demand.

Clarifying Questions

Question	Answer
Scale?	150M MAU; 25M trips/day; 6M concurrent online drivers at peak
Geography?	Global; partition operations by city / metro region
Match latency?	Driver offer within 2s of rider request; p99 < 5s
Pricing?	Real-time surge multiplier per region, shown pre-request
Routing?	Custom routing engine (OSRM-like); ETAs from live telemetry
Payments?	Charged after trip; multiple methods; tipping post-trip
Offline driver app?	Buffer location pings; graceful timeout on dispatch

02

Requirements

Functional and non-functional

Functional Requirements

Requirement	Details
Driver location	Driver app pings location every 4 sec; geo-indexed
Rider request	Rider taps 'request' → server matches a nearby driver
Dispatch	Offer trip to top-scored driver; 15s timeout; fall through
Trip lifecycle	States: requested → accepted → en_route → at_pickup → in_progress → completed
Surge pricing	Multiplier per H3 cell, computed from supply / demand
ETA	Live ETA to pickup and to destination from routing engine
Payments	Auto-charge after trip end; receipts; tipping window

Non-Functional Requirements

Requirement	Target
Match latency	p50 < 1s, p99 < 5s from request to driver offer
Availability	99.99% for trip lifecycle; 99.9% for analytics
Geo-index freshness	Driver location TTL ≤ 10s; query freshness < 5s
Throughput	1.5M location updates/sec at peak; ~300 trips/sec global
Consistency	Strong for trip state machine; eventual for pricing
Scale	Horizontally scalable per city/region shard

03

Capacity Estimation

Math for scale

Traffic Estimation

- MAU: **150M**; DAU $\approx$ 30–40M
- Trips/day: **25M** globally $\rightarrow 25M / 86,400s \approx$ **290 trips/sec** avg
- Peak factor 3–4 $\times \rightarrow$ **~1,000–1,200 trip requests/sec** at peak
- Concurrent online drivers at peak: **6M**
- Driver location ping every **4 sec** $\rightarrow 6M / 4 =$ **1.5M location updates/sec**

Storage Estimation

- Location ping payload: driver_id (8B) + lat/lng (16B) + h3_cell (8B) + ts (8B) + bearing/speed (8B) $\approx$ 50–80 B
- Hot location store: only the **latest** ping per driver in Redis ($6M \times \sim 200B \approx$ **1.2 GB**)
- Trip record: ~ 2 KB; $25M/day \times 2$ KB = **50 GB/day** trip data
- Trip events to Kafka: ~ 10 events/trip $\times 25M = 250M$ events/day, ~ 500 GB/day raw
- Annual trip storage (compressed in OLAP): **~5–10 TB/year**

Geo-index Working Set

- H3 res 9 cell ≈ 0.1 km²; a busy metro has $\sim 10K$ active cells
- Per cell, store list of driver_ids currently in the cell (avg 5–50)
- Total geo-index entries: **6M drivers**, lookup by H3 cell key
- Memory for indexed structures: **~3–5 GB Redis** (latest pos + cell membership)

KEY

Key Numbers

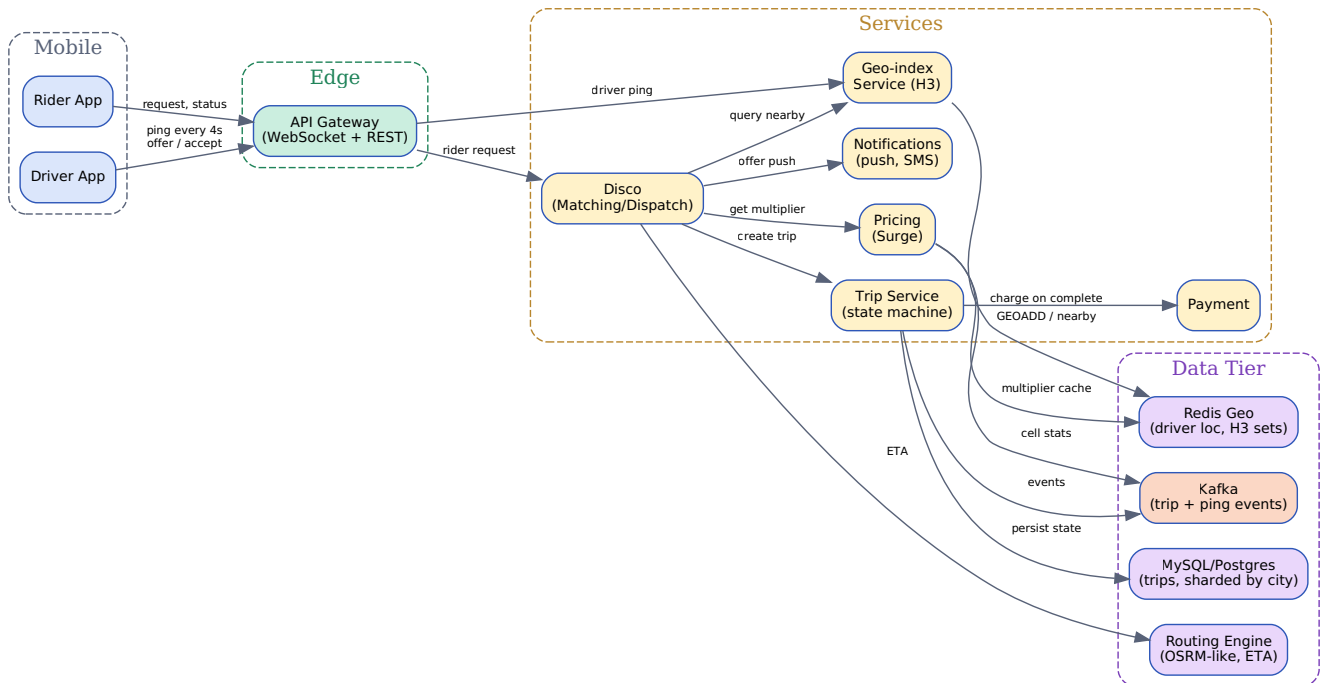
150M MAU · 25M trips/day · 6M concurrent drivers · **1.5M location updates/sec** at peak · $\sim 1,000$ trip requests/sec at peak · ~ 1.2 GB hot driver-location set.

04

High-Level Architecture

System overview

The platform splits into **mobile clients** (rider and driver apps), an **API gateway** for ingress, a set of **matching and trip services** (Uber's *Disco* dispatch, Geo-index, Trip, Pricing), and a **data tier** (Redis geo, MySQL/Postgres sharded by city, Kafka for events, and the routing engine). Each city/metro is a sharding unit; cross-region traffic is rare.



Rider/Driver mobile apps → API gateway → Disco (matching) coordinates Geo-index, Trip, Pricing, Payments, Notifications.
Data tier: Redis geo, MySQL trips, Kafka events, Routing engine.

Architecture Highlights

- **API Gateway:** long-lived WebSocket per app for low-latency push (offers, status) and REST for control
- **Disco:** matching brain — turns rider request + nearby drivers into a sequence of offers
- **Geo-index:** stateful service over Redis; converts (lat,lng) ↔ H3 cell; answers 'who is near?'
- **Trip Service:** owns the state machine; durable in MySQL; saga across pricing, payment, notify
- **Pricing:** consumes supply/demand stats per H3 cell; emits surge multiplier
- **Routing Engine:** custom graph routing over OpenStreetMap; updated by driver telemetry
- **Kafka:** backbone for ping/trip/payment events feeding analytics, ML, fraud, surge

05

Geo-Indexing with H3

Hex grid over the planet

Why a Hex Grid?

- **Equidistant neighbors:** a hexagon has 6 neighbors all the same distance apart; squares have 4 close + 4 diagonal (different distances)
- **Uniform area:** H3 cells of a given resolution have nearly the same area, regardless of latitude
- **Hierarchical:** each cell has parent/children at different resolutions for zoom
- **Simple ring queries:** 'k-ring' returns all cells within k steps — natural for radius search

Resolution Choice

H3 res	Avg cell area	Use
res 6	~36 km ²	Surge zones, regional supply/demand views
res 7	~5 km ²	Default surge zone size; metro regions
res 8	~0.7 km ²	Mid-zoom dispatch context
res 9	~0.1 km ²	Driver-location bucket; primary dispatch index
res 10	~0.015 km ²	Pickup-precise; used for pickup snapping

H3 vs S2 vs Quadtree

Property	H3 (hex)	S2 (sphere cells)	Quadtree
Cell shape	Hexagon	Quadrilateral on sphere	Square (axis-aligned)
Neighbor distance	Uniform (6 neighbors)	Approx uniform (8 neighbors)	Non-uniform (4+4 diag)
Area uniformity	High (hierarchical)	High (Hilbert-curve)	Distorted near poles, equator OK
Parent / child	Aperture-7 (not exact partition)	Exact 4-way subdivision	Exact 4-way subdivision
Library	Uber H3	Google S2	Many implementations

Property	H3 (hex)	S2 (sphere cells)	Quadtree
Best for	Mobility, surge, dispatch	Spatial joins, indexing	Simple range queries

INSIGHT

Why H3 Wins for Dispatch

Equidistant neighbors mean a 'k-ring' lookup gives a true radius, not a stretched square. Uniform area makes per-cell supply/demand metrics fair across a city. Aperture-7 hierarchy is good enough for analytics with small overlap correction.

06

Driver Location Pipeline

Ingesting 1.5M pings/sec

End-to-end Flow

1. Driver app sends POST /ping over WebSocket every 4 sec with (lat, lng, bearing, speed)
2. API Gateway forwards ping to Geo-index Service partition for that driver's region
3. Geo-index computes `h3_cell = h3.geo_to_h3(lat, lng, res=9)`
4. Update Redis: `GEOADD drivers:active <lng> <lat> <driver_id>` and `SADD cell:<h3_cell> <driver_id>`
5. Set TTL on driver key (10 sec); stale drivers fall out automatically
6. Async: emit ping to Kafka topic `driver_pings` for surge, fraud, ETA telemetry

Storage: driver_location Schema

```
-- Redis (hot, primary)
GEOADD drivers:active <lng> <lat> <driver_id>
HSET driver:<driver_id> h3 <h3_cell> ts <epoch_ms> status 'available'
EXPIRE driver:<driver_id> 10
SADD cell:<h3_cell> <driver_id>

-- MySQL (cold, append-only history; partition by day, sharded by city)
CREATE TABLE driver_location (
  driver_id BIGINT NOT NULL,
  city_id INT NOT NULL,
  lat DOUBLE NOT NULL,
  lng DOUBLE NOT NULL,
  h3_res9 BIGINT NOT NULL, -- H3 cell ID
  bearing SMALLINT,
  speed_kph SMALLINT,
  ts TIMESTAMP(3) NOT NULL,
  PRIMARY KEY (driver_id, ts),
  KEY idx_h3 (h3_res9, ts)
) PARTITION BY RANGE (TO_DAYS(ts));
```

Why Redis (and not MySQL) for the Hot Index

- **1.5M writes/sec:** Redis handles 100K+ ops/sec per node; 16-node cluster sharded by region absorbs load
- **Sub-ms reads:** nearby-driver queries must return in < 5 ms for sub-second match latency
- **TTL primitives:** stale drivers expire automatically; no scrubbing job

- **Native geo ops:** GEOADD / GEOSEARCH; we combine with H3 sets for hex-aware lookup

07

Disco: Matching & Dispatch

Rider request to driver offer

End-to-end Flow

1. Rider taps 'request' → POST /trip/request with (pickup_lat, pickup_lng, dest)
2. Trip Service creates trip in state requested; emits trip.requested
3. Disco computes pickup_h3 = h3.geo_to_h3(lat, lng, 9) and queries Geo-index for rings 1, 2, 3
4. Filter candidates: online + available + service-tier match + not currently on a trip
5. Score each candidate by ETA + driver rating + acceptance rate (see pseudocode below)
6. Send offer to top driver via push (WebSocket); start **15 s timeout** timer
7. If accepted → trip transitions to accepted; if declined / timeout → fall through to next candidate
8. On accept: notify rider, lock driver, push ETA to pickup

Matching Pseudocode

```
def match(rider_request):
    pickup_h3 = h3.geo_to_h3(rider_request.lat,
                             rider_request.lng, res=9)

    # Expand outward until we have enough candidates
    candidates = []
    for ring in range(0, 4):          # rings 0..3
        cells = h3.k_ring(pickup_h3, ring)
        for cell in cells:
            for driver_id in redis.smembers(f'cell:{cell}'):
                d = driver_state(driver_id)
                if d.online and d.available \
                    and d.tier == rider_request.tier:
                    candidates.append(d)
    if len(candidates) >= 20:
        break

    # Score: lower ETA, higher rating, higher accept rate
    def score(d):
        eta = routing.eta(d.location, rider_request.pickup)
        return (-eta * 1.0
                + d.rating * 30
                + d.accept_rate * 20)

    candidates.sort(key=score, reverse=True)
```

```
# Sequential offer with 15s timeout each
for d in candidates[:5]:
    if offer_with_timeout(d, rider_request, t=15):
        return d                # accepted
raise NoDriverAvailable()
```

Sync Dispatch vs Batch Dispatch

Aspect	Synchronous (immediate)	Batched (every 1–2s)
Latency to first offer	Lowest (≈ 1 s)	Adds the batch window (1–2 s)
Match quality	Greedy: first decent driver wins	Globally optimal — uses Hungarian/min-cost flow
Best for	Sparse demand, off-peak	Dense urban areas during surge
Failure mode	Locally suboptimal pairings	Rider feels app is 'thinking'
Uber's choice	Default greedy with score	Batch when supply > demand and cell is hot

TIP

Why Sequential Offers (not 'broadcast')?

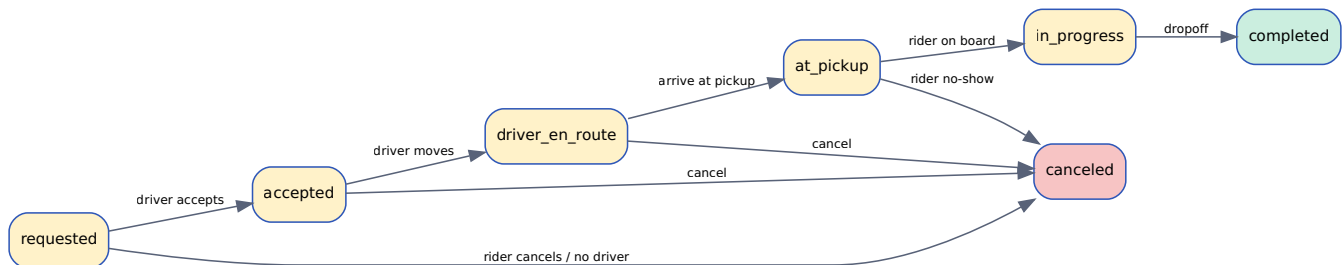
Broadcasting to many drivers at once causes contention (multiple accepts) and unfair allocation. Sequential offers with a 15 s timeout give a fair, deterministic flow and let drivers self-select.

08

Trip Lifecycle & State Machine

Idempotent transitions across services

States



Trip state machine. Transitions are idempotent; each is recorded as an event in Kafka and a row in trip_events.

Trips Table Schema

```

CREATE TABLE trips (
  trip_id      BIGINT      PRIMARY KEY,
  city_id      INT         NOT NULL,      -- shard key
  rider_id     BIGINT      NOT NULL,
  driver_id    BIGINT,
  state        ENUM('requested', 'accepted', 'driver_en_route',
                    'at_pickup', 'in_progress', 'completed',
                    'canceled') NOT NULL,

  pickup_lat   DOUBLE,
  pickup_lng   DOUBLE,
  pickup_h3_r9 BIGINT,
  dest_lat     DOUBLE,
  dest_lng     DOUBLE,
  base_fare    DECIMAL(10,2),
  surge_mult   DECIMAL(4,2),
  total_fare   DECIMAL(10,2),
  requested_at TIMESTAMP(3) NOT NULL,
  accepted_at  TIMESTAMP(3),
  completed_at TIMESTAMP(3),
  state_version INT         NOT NULL,      -- optimistic concurrency
  KEY idx_driver (driver_id, requested_at),
  KEY idx_rider  (rider_id, requested_at)
);

CREATE TABLE trip_events (
  trip_id    BIGINT,
  event_seq  INT,
  event_type VARCHAR(32),

```

```

payload      JSON,
ts           TIMESTAMP(3),
PRIMARY KEY (trip_id, event_seq)
);

```

Idempotency

- **state_version:** guarded transition `UPDATE ... WHERE state_version = ?` rejects double-apply
- **idempotency_key:** client-supplied UUID on POSTs (request, accept, complete) — server dedups
- **Outbox pattern:** trip_events row written in same Tx as trips update; relay publishes to Kafka
- **Saga:** trip completion fans out to Pricing → Payment → Notify; compensating actions on failure

Trip Saga Across Services

Step	Service	Compensation if it fails
1. Reserve driver	Disco	Release lock; offer next driver
2. Create trip row	Trip	Mark canceled with reason
3. Compute fare (surge × distance)	Pricing	Use last-known multiplier
4. Charge payment	Payment	Retry; fall back to deferred billing
5. Push receipt	Notify	Best-effort; log if undelivered

09

Surge Pricing

Real-time supply/demand balance

Surge multiplies the base fare in zones where rider requests outpace available drivers. The multiplier is broadcast to rider apps **before** a request, so the rider sees the price up front and the system can shape demand.

Per-Cell Computation (H3 res 7 $\approx$ 5 km²)

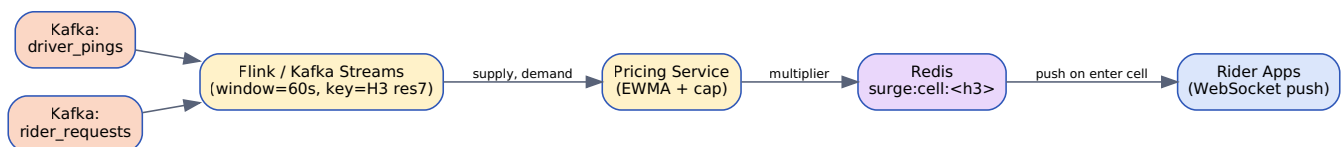
1. Aggregate per cell, per minute: `requests = count(rider.requested)`, `supply = count(distinct online available drivers)`
2. Compute imbalance ratio $r = \max(1, \text{requests} / \max(\text{supply}, 1))$
3. Smooth with EWMA: `multiplier =  $\alpha \cdot \text{clamp}(r, 1.0, 5.0) + (1-\alpha) \cdot \text{prev}$` ($\alpha=0.3$)
4. Publish to Redis `surge:cell:<h3> = multiplier` with TTL 90 s
5. Rider app polls or receives push when entering a new cell

Pricing Formula

```
fare = base_fare
      + per_minute * trip_minutes
      + per_km      * trip_km
      + booking_fee

fare *= surge_multiplier(pickup_h3_res7)  # capped at 5.0
fare = max(fare, min_fare)
```

Surge Stream



Pricing service consumes Kafka topics (rider requests, driver pings) windowed by H3 cell, emits surge multiplier to Redis, broadcast to rider apps.

Safeguards

- **Hard cap:** multiplier ≤ 5.0 ; emergencies (storms, disasters) $\rightarrow$ cap at 1.0 manually
- **Smoothing:** EWMA prevents oscillation; minimum dwell of 60 s before raising
- **Show before tap:** rider sees price pre-request; reduces complaints and surprises

- **Driver-side:** heatmap of high-surge cells nudges supply to where it's needed

Routing Engine

- **Graph:** road network from OpenStreetMap; nodes = intersections, edges = road segments with travel-time weights
- **Algorithm:** contraction hierarchies (CH) or A* with landmarks; ETAs in < 10 ms per query
- **Custom build:** Uber's in-house engine (similar to OSRM / Valhalla) tuned for batch ETA at dispatch time
- **Two ETA queries per trip:** driver → pickup, pickup → destination

Real-Time Traffic Updates

1. Driver pings include speed and bearing every 4 sec
2. Stream (Kafka → Flink) bins pings to OSM edges; computes p50 / p85 speed per edge per 5-min window
3. Routing engine reloads edge weights every 1–5 minutes
4. Historical patterns (time-of-day, day-of-week) are blended for cold edges with no recent telemetry

ETA Accuracy

Layer	Source	Update freq
Static graph	OSM nightly build	Daily
Historical speed	Past 90 days, by hour-of-week	Weekly
Live speed	Driver telemetry, last 5 min	Every 1–5 min
ML correction	Gradient-boosted residual on top	Per-trip inference

INSIGHT

Why Custom Routing?

Off-the-shelf routers are tuned for human navigation (turn-by-turn). Dispatch needs *millions of ETA queries per minute* for matching, with low p99 latency. A purpose-built engine with pre-contracted graphs and bulk-query APIs hits sub-10 ms per ETA at scale.

11

Storage & Sharding

Hot Redis, sharded SQL, Kafka events

Storage Tiers

Data	Store	Reason
Latest driver location	Redis (geo + hash)	Sub-ms reads, native geo, TTL
H3 cell membership	Redis sets	O(1) add/remove; quick k-ring lookup
Trip records (truth)	MySQL/Postgres sharded by city	ACID for state machine; relational queries
Trip / ping events	Kafka	Immutable log; fans out to surge, ML, fraud, analytics
Trip history (cold)	Cassandra / S3 + Parquet	Cheap, append-only; OLAP queries
Surge multiplier	Redis (90 s TTL)	Hot read path; tolerates short staleness

Sharding by City

- **Shard key:** `city_id` (San Francisco, NYC, London ...) — most queries are local
- **Hot cities split:** NYC, SF, London get their own dedicated MySQL clusters
- **Cold cities pooled:** dozens of small markets share a cluster
- **Cross-city trip:** rare (airport runs); handled by writing to origin city, mirroring to destination
- **Per-region datacenters:** US-East, US-West, EU, APAC; matching is always intra-DC

Replication & Consistency

- **Trip table:** primary + 2 sync replicas per shard; failover via Orchestrator
- **Read-after-write:** trip mutations always read from primary (low volume)
- **Redis geo:** primary + replica per region; data is rebuildable from pings within 10 sec
- **Kafka:** RF=3, min ISR=2; cross-region mirroring for analytics

TIP

Cell Hashing Across Redis Nodes

Shard the geo-index by H3 parent cell (res 4 or 5) using consistent hashing. A k-ring query at res 9 stays within one or two Redis nodes. Adding capacity moves only $1/N$ of cells.

12

Failure Modes & Recovery

What can go wrong

Failure	Impact	Detection	Mitigation
Driver app loses connectivity	Pings stop; driver disappears from cell	TTL expiry on driver: key	Graceful timeout (10 s); offer fall-through to next candidate; re-add on reconnect
Rider app loses connectivity mid-trip	Status updates not received	WebSocket disconnect	Trip continues server-side; reconcile state on reconnect via /trip/<id>
Surge runaway (oscillation / spike)	Multiplier flapping; rider trust hit	Multiplier change rate alert	Hard cap 5.0; EWMA smoothing; manual emergency cap to 1.0
Payment failure post-trip	Trip completed but unpaid	Payment async error	Saga retry with backoff; mark trip 'unpaid' and recover via batch settlement
Redis geo node down	Some cells lose drivers; partial blackouts	Health check, miss-rate spike	Failover to replica; data rebuilds from pings within 10 s
MySQL shard down	Trips in that city stall	Connection timeout	Auto-promote replica; block new requests; queue updates
Disco region overload	Match latency spikes	p99 alert > 5 s	Shed load: queue requests; raise surge to throttle demand
Kafka cluster lag	Surge stale; analytics lag	Consumer-lag alert	Use last-known surge in Redis; backfill events on recovery

WARNING

Graceful Degradation

If the matching engine is degraded, prioritize active trips: never break an in-progress trip's state machine. New requests can be delayed or shown 'no drivers available' — that's recoverable. A lost in-progress trip is not.

13

Design Trade-offs

Decisions and rationale

Decision	Choice	Trade-off
Geo grid	H3 hex (Uber)	Equidistant neighbors and uniform area beat S2 / quadrees for radius search; aperture-7 hierarchy is approximate.
Hot location store	Redis (in-memory)	Sub-ms reads; loses 10 s of data on node death (rebuilds from pings). MySQL / SQL would be too slow at 1.5M writes/sec.
Dispatch model	Sequential offers, score-ranked	Lower latency to first offer; locally suboptimal pairings. Batch dispatch optimizes globally but adds 1–2 s.
Match latency vs quality	Greedy with score (rating, accept_rate, ETA)	Fast and fair. Pure ETA-min would worsen driver fairness; pure rating-max would hurt latency.
Sharding	By city_id	Local queries dominate; trivially scales out per market. Cross-city trips need extra wiring.
Surge cap	Multiplier capped at 5.0	Limits gouging and runaway spikes; on real shortages, demand exceeds supply (some riders unhappy).
Trip state	Strong consistency in MySQL	Prevents double-acceptance and lost trips; lower throughput than Cassandra for the same row.
Routing	Custom OSRM-like engine	10 ms ETAs at scale; engineering investment vs using Google Maps API (cost + rate-limit pain).
Driver ping frequency	4 sec	Smooth tracking and accurate ETAs; 1 sec would 4× backend load. 30 sec would degrade dispatch.
Regional dispatch	Intra-DC matching only	Sub-second p99; cross-region rider/driver pairs are rare.

14

Scalability & Distribution

Going from one city to a planet

Horizontal Scaling

- **API Gateway:** stateless; sticky WebSocket sessions per device, scale on connection count
- **Geo-index:** partition by H3 parent cell (res 4); each partition owned by one node
- **Disco:** stateless; sharded by pickup city; horizontally scaled by request rate
- **Trip Service:** sharded by city_id; co-located with MySQL primary for that city
- **Pricing:** Flink job sharded by H3 res-7 key; output to Redis surge:cell:

Multi-Region Layout

- **Region per continent:** US-East, US-West, EU, APAC, LATAM
- **Local primaries:** matching, trip, pricing all run intra-DC for sub-second p99
- **Driver and rider stickiness:** their session lives in the region serving their city
- **Cross-region async:** only analytics (Kafka mirror) + cross-city tracking

Capacity Headroom

- **Peak driver pings:** 1.5M/sec → ~16 Redis primaries × 100K ops/sec each
- **Peak trip requests:** ~1,200/sec → 50–100 Disco workers handle easily
- **Trip writes:** ~3,000/sec MySQL inserts globally (multi-event per trip) → comfortably within sharded MySQL
- **Headroom:** design for 3× peak; New Year's Eve hits ~2.5× a typical Friday

INSIGHT

City-as-a-Shard

Treating each city as an independent shard gives near-linear scalability and clean blast radius. A bug or outage in one city doesn't impact others, and onboarding a new city is a configuration step, not an architectural one.

Designing Uber is a flagship system-design interview. The key is to lead with the geo-indexing decision, drive cleanly through dispatch, and finish with surge + the trip state machine. Don't drown in payments.

45-Minute Interview Outline

1. **Intro (2 min):** 150M MAU, 25M trips/day, 6M concurrent drivers, ping every 4 s → 1.5M updates/sec
2. **Capacity (4 min):** 1.5M location writes/sec, ~1,200 trip req/sec at peak, ~50 GB/day trip data
3. **Geo-index (6 min):** H3 hex, res 9 dispatch + res 7 surge; hex vs square vs quadtree
4. **Architecture (3 min):** Mobile → Gateway → Disco / Geo / Trip / Pricing / Payment / Notify → Redis / MySQL / Kafka / Routing
5. **Dispatch deep dive (8 min):** k-ring query, score by ETA + rating + accept rate, sequential offers with 15 s timeout
6. **Trip lifecycle (6 min):** state machine, idempotent transitions, saga across services, outbox + Kafka
7. **Surge (5 min):** per-cell supply/demand, EWMA, cap at 5×, broadcast pre-request
8. **Storage & sharding (4 min):** Redis hot, MySQL by city, Kafka events; H3-aware Redis sharding
9. **Failures & trade-offs (5 min):** driver disconnect, surge runaway, payment retry; sync vs batch dispatch
10. **Wrap (2 min):** what you'd build first, what you'd defer (international payments, fraud)

Key Talking Points

- “1.5M location updates/sec” — drives Redis geo + city sharding decision
- “H3 hex grid: equidistant neighbors, uniform area” — explicit reason over S2 / quadtree
- “k-ring 1, 2, 3 + score by ETA + rating + accept rate” — concrete dispatch algorithm
- “15 s offer timeout, sequential fall-through” — fairness + low contention
- “Trip state machine with idempotent transitions, outbox + Kafka” — distributed correctness
- “Surge per H3 res 7 cell, EWMA-smoothed, capped at 5×, broadcast pre-request” — economics + UX
- “City as a shard” — clean blast radius, near-linear scalability

Common Follow-ups & Answers

- **Q: Why H3 over S2?** A: Hex cells have equidistant neighbors and uniform area, which makes 'k-ring' a true radius search and per-cell metrics fair. S2 is great for indexing but uneven for radius queries.
- **Q: What if two riders request the same driver?** A: Disco locks the driver in Redis on offer (SETNX with TTL). The second match query skips locked drivers; on accept, the lock is committed.

- **Q: How do you avoid surge runaway?** A: Hard cap (5×), EWMA smoothing, minimum dwell time. In emergencies, manual override caps at 1×.
- **Q: How do you handle driver app loss-of-connection?** A: TTL on driver key in Redis (10 s); driver falls out of cell after one missed ping interval. On reconnect they re-register.
- **Q: How do payments not block trip completion?** A: Trip completion is a saga: state goes to completed first, payment is async with retries, and unpaid trips are settled in batch.
- **Q: Sync vs batch dispatch?** A: Default sync (greedy, lowest latency). Switch to batch (1–2 s window, min-cost flow) in dense, high-supply cells where global optimality wins.
- **Q: Why not just GEOADD without H3?** A: GEOSEARCH alone gives a circle, but per-cell metrics (surge, supply heatmaps) need a stable, area-uniform tessellation. H3 + Redis sets is the combo.

KEY

Memorize These Numbers

150M MAU · 25M trips/day · 6M concurrent drivers · ping every 4 s → **1.5M location writes/sec** · ~1,200 trip req/sec at peak · H3 res 9 $\approx$ 0.1 km² (dispatch), res 7 $\approx$ 5 km² (surge) · 15 s offer timeout · surge cap 5×.